

array

October 23, 2025

Objective:

To become comfortable with NumPy arrays and basic operations.

Like other Python libraries, must be explicitly imported into a Python script or interactive session to make its functions and objects accessible.

Python uses namespaces to prevent naming clashes. A namespace is a container that holds a collection of identifiers like variable and function names.

Without explicit imports, if two different libraries used the same function name, they would conflict.

```
[3]: # np is an alias or alternative name.  
import numpy as np
```

```
[5]: # .array() function accepts a list as an argument.  
arr1 = np.array([1, 2, 3, 4, 5])
```

```
[7]: # Arrays in NumPy are instance of the ndarray class.  
type(arr1)
```

```
[7]: numpy.ndarray
```

```
[8]: type([1, 2, 3, 4, 5])
```

```
[8]: list
```

```
[47]: # If the dtype parameter is left out of the function call,  
# the default type will be float.  
  
# int, str, and bool are just some of the data types  
# can be assigned to the dtype parameter.  
matrix1 = np.zeros((2, 3), dtype=bool, order='F')
```

```
[48]: matrix1
```

```
[48]: array([[False, False, False],
            [False, False, False]])
```

```
[49]: type(10.)
```

```
[49]: float
```

```
[50]: # name_of_matrix[row_range, column_range]
first_row = matrix1[0, :]
```

```
[29]: first_row
```

```
[29]: array([0, 0, 0])
```

The code demonstrates how the memory layout (C-order vs F-order) of a NumPy array affects computational performance when performing operations along different axes.

```
[30]: import time

# Create a large 2D array
shape = (5000, 5000)

# Create a C-order (row-major) array
c_array = np.arange(shape[0] * shape[1], dtype=np.int32).reshape(shape,
    ↪order='C')

# Create an F-order (column-major) array
f_array = np.arange(shape[0] * shape[1], dtype=np.int32).reshape(shape,
    ↪order='F')

# Test performance for row-wise sum (fast for C-order)
print("Timing row-wise sum:")
start_time = time.time()
np.sum(c_array, axis=1) # Summing along rows is fast for C-order
print(f"C-order time: {time.time() - start_time:.4f}s")

start_time = time.time()
np.sum(f_array, axis=1) # Summing along rows is slower for F-order
print(f"F-order time: {time.time() - start_time:.4f}s")
print("-" * 20)

# Test performance for column-wise sum (fast for F-order)
print("Timing column-wise sum:")
start_time = time.time()
np.sum(c_array, axis=0) # Summing along columns is slow for C-order
print(f"C-order time: {time.time() - start_time:.4f}s")
```

```

start_time = time.time()
np.sum(f_array, axis=0) # Summing along columns is fast for F-order
print(f"F-order time: {time.time() - start_time:.4f}s")

```

Timing row-wise sum:

C-order time: 0.0934s

F-order time: 0.0817s

Timing column-wise sum:

C-order time: 0.0767s

F-order time: 0.0744s

A matrix with three rows and three columns.

```
[43]: matrix2 = np.ones((3, 3))
```

```
[44]: matrix2
```

```
[44]: array([[1., 1., 1.],
            [1., 1., 1.],
            [1., 1., 1.]])
```

```
[2]: for i in range(1, 11, 1):
      # print(i)

      if i != 10:
          # Positional argument: 1
          # Keyword argument: end="\n"
          print(i, end=", ")
      else:
          print(i)

```

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

```
[6]: arr2 = np.array([0, 2, 4, 6, 8])
```

```
[8]: arr2
```

```
[8]: array([0, 2, 4, 6, 8])
```

The following produces the same ndarray as the above.

```
[9]: # np.arange(start, stop (exclusive), step)
arr3 = np.arange(0, 10, 2)
```

```
[10]: arr3
```

```
[10]: array([0, 2, 4, 6, 8])
```

Element-wise comparison: Compares each element of arr2 with the corresponding element in arr3.

```
[11]: arr2 == arr3
```

```
[11]: array([ True,  True,  True,  True,  True])
```

The subsequent line returns False because arr2 and arr3 are two different objects.

```
[12]: arr2 is arr3
```

```
[12]: False
```